

Ingeniería de software 2

Clase 1

Proceso de software

Conjunto de actividades y resultados asociados que producen un producto de software.

El modelo de proceso de software es una representación abstracta de un proceso de software.

Las actividades fundamentales de los modelos de proceso serán

- Especificaciones del software
 - Técnicas de elicitación.
 - Especificación de requerimientos.
- Desarrollo del software.
- Validación del software.
- Evolución del software.

La **comunicación** es el proceso más importante de la interacción humana, una necesidad personal y la forma más completa de crear la realidad. Las personas comunicamos constantemente, tanto con las palabras (comunicación verbal) como con nuestra actitud y comportamiento (comunicación no verbal).

Elicitación de requerimientos

Es el proceso de adquirir todo el conocimiento relevante necesario para producir un modelo de los requerimientos de un dominio del problema

Objetivos

- Conocer el dominio del problema para poder comunicarse con clientes y usuarios y entender sus necesidades.
- Conocer el sistema actual.
- Identificar las necesidades, tanto explícitas como implícitas, de clientes y usuarios y sus expectativas sobre el sistema a desarrollar.

Algunas técnicas de elicitación de requerimientos serán

Muestreo de la documentación, los formularios y los datos existentes

Es la recolección de hechos a partir de la documentación existente. Los documentos que se expondrán serán: organigramas, notas internas, minutas, registros contables, solicitudes de proyectos de sistemas anteriores y permitirán conocer el historial que origina el proyecto.

Observación del ambiente de trabajo

El analista se convierte en observador de las personas y actividades con el objeto de aprender acerca del sistema. Se seguirán los siguientes lineamientos de la observación

- Determinar quién y cuándo será observado.
- Obtener el permiso de la persona y explicar el porqué será observado.
- Mantener bajo perfil.
- Tomar nota de lo observado.

- Revisar las notas con la persona apropiada.
- No interrumpir a la persona en su trabajo.

Visitas al sitio

— agregar info inge 1.

Cuestionarios

— agregar info inge 1.

Entrevistas

— agregar info inge 1.

Los pasos a seguir serán

1. Leer los antecedentes poniendo atención en el lenguaje, buscará el vocabulario común y es necesario para poder entender al entrevistado. Será necesario establecer los objetivos de la entrevista teniendo en cuenta los antecedentes leídos.
2. Usualmente los directivos suelen proporcionar una visión general, mientras que los futuros usuarios dan un más detallada. Por lo tanto, se seleccionan los entrevistados con el objetivo de minimizar el número de entrevistas los cuales deben conocer el objetivo y las preguntas que se le realizarán.
3. Planificación de la entrevista y preparación del entrevistado: establecer fecha, hora, lugar y duración de cada entrevista de acuerdo con el entrevistado.
 - Informar al entrevistado el tema a tratar antes de la reunión.
 - Definir un "Guion de entrevista".
 - Se deben evitar preguntas sesgadas o con intención, amenazantes o críticas.
 - Usar lenguaje claro y conciso.
 - No incluir opinión como parte de la pregunta.
 - Evitar preguntas largas y complejas.
4. Selección del tipo de preguntas a usar y su estructura
Hay diferentes organizaciones para las entrevistas, serán
 - Piramidal.
 - Embudo.
 - Diamante.

Planeación conjunta de requerimientos (JRP)

Proceso mediante el cual se conducen reuniones de grupo altamente estructurados con el propósito de analizar problemas y definir requerimientos.

Requiere de extenso entrenamiento, reduce tiempo de exploración de requisitos, amplia participación de los integrantes y se trabaja sobre lo que se va generando.

Brainstorming

Técnica para generar ideas al alentar a los participantes para que ofrezcan tantas ideas como sea posible en un corto tiempo sin ningún tipo de análisis hasta que se hayan agotado las ideas.

Clave para resolver la falta de consenso entre usuarios y es útil combinarlo con la toma de decisiones, ayuda a entender del dominio del problema y encara la dificultad del usuario para transmitir, ayuda a entender al usuario y al

analista.

Comunicación con presentaciones tipo PPT

Se pueden incluir PPTT para comunicar. Kawasaki recomienda algunas reglas para realizarlo,

- La presentación debe contar como máximo 10 diapositivas.
- No sobrepasar los 20 minutos de presentación.
- No usar tipografías con cuerpo menor a 30.
- Buscar simplificar información.

Se recomienda además

- No leer la presentación al pie de la letra.
- Usar la zona de notas de la diapositiva.
- No mirar la pantalla exponiendo.
- Colocarse en el centro del escenario.
- Usar tamaño legible, 32 si es posible.
- No animar textos.
- Usar letra legible.
- Usar fondos claros y sencillos.
- Diseñar diapositivas para entenderse en 3 segundos.
- Cerrar la presentación con: sesión de preguntas, resumen de lo expuesto o invitación a contactarse.

Algunos detalles estéticos serán

- Colores: usar colores oscuros para los textos y pasteles pálidos para el fondo.
- Textos: mantener el formato de fuente (tamaño, tipo, etc).
- Imágenes: usar visuales para explicar información que es difícil de comprender.
- Transiciones y animaciones: no deben ser excesivas y tienen que tener cierto sentido.
- Formato de las diapositivas: el formato de las diapositivas debe ser el de un esquema, en el que cada párrafo representará una idea.

La presentación debe ser llamativa visualmente, ofrecer algo distintivo o novedoso por lo que debe ser invitada a ser visualizada y no debe faltar

- Nombre del proyecto.
- Nombre de la empresa que lo lleva adelante.
- Objetivos.
- Características o funcionalidades principales.
- Técnicas de recolección de datos que se emplean.
- Contacto.

Video tipo elevator pitch

Elevator pitch es un mensaje corto de aproximadamente un minuto de duración para contar tu proyecto de manera diferenciadora. El cual sigue los pasos de

1. Afirmar o preguntar sorprendentemente para llamar la atención.
2. Contar quién sos.
3. Problemas o necesidades que cubrís.
4. Solución aportada.
5. Beneficio principal que brindás.
6. Por qué el proyecto es idóneo.
7. Terminar una llamada a la acción final.

Otros tips para el video será

- Prestar atención a la comunicación NO verbal, las personas del video deben sonreír, usar manos para contar ideas o puntos importantes, mover las manos en referencia a lo que se está contando.
- Verbal: poner diferentes énfasis en la voz y hacerlo en los momentos adecuados.

Requerimientos

Los requerimientos es una característica de algo que hace el sistema con el objetivo de satisfacer necesidades del sistema en sí.

Tipos de requerimientos

- Funcionales: definen el comportamiento y las tareas del sistema. A la hora de definirlos es importante mantener el equilibrio entre la generalidad y el detalle.
- No funcionales: define aspectos que son deseables desde el punto de vista del usuario, incluye restricciones como reglas, tiempos de respuesta, características de usabilidad.

Especificación de requerimientos

La *especificación de requerimientos* es un documento que es dirigido a los usuarios para describir características de un sistema propuesto.

Esta descripción es el medio de comunicación que recoge visión general, cualitativa y cuantitativa de las características del sistema.

Descripción del sistema- IEEE 1362

Ofrece un formato y contenidos para confeccionar las descripciones del sistema. No da técnicas exactas si no líneas generales a respetarse, a modo de apoyo.

Requerimientos del software es un documento también denominado SRS que es una especificación que realiza un determinado producto de software, programa o conjunto de programas en un determinado entorno.

El documento de especificación de requerimientos puede desarrollarlo personal representativo de la parte desarrolladora, o de la parte cliente; se recomienda que lo hagan ambas partes.

// AMPIAR SRS

IEEE 830- SRS

Las especificaciones de requisitos de software (SRS) son documentos detallados que describen de manera exhaustiva los requisitos y características de un sistema de software. Estas especificaciones actúan como un contrato entre los clientes, usuarios y desarrolladores, estableciendo qué funcionalidades debe tener y cómo debe comportarse.

El SRS es una especificación para un producto de software particular el cual es redactado por uno o más representantes del equipo del desarrollo y del cliente.

El alcance se refiere a los límites y fronteras de un proyecto. Describirlo implica definir qué incluirá o lo que no, así como establecer cuál es el ámbito del proyecto, funcionalidades y características se abordarán.

Características de un buen SRS

- Correcto: cada requisito declarado se encuentra en el software.
- No ambiguo: cada requisito declarado tiene una sola interpretación.
- Completo: se reconoce cualquier requisito externo impuesto por una especificación del sistema.
- Consistente: si un SRS no está de acuerdo con algún documento del nivel superior, como una especificación de requerimientos de sistema.
- Priorizado: da importancia a cada requerimiento.
- Consistente: cada requisito tiene que ser comprobable, significa que un proceso o una máquina puede verificar que el producto reúne el requisito.
- Modificable: un SRS debe permitir que con la estructura y estilo que tenga pueda modificarse en caso de ser necesario.
- Trazabilidad: claridad del origen de cada requerimiento y su trazabilidad hacia futuros requerimientos.

Para un buen SRS, tiene que ser elaborado en colaboración de todas las partes interesadas para un acuerdo sólido y equitativo.

El SRS debe evolucionar de manera paralela al desarrollo de software, registrando todos los cambios que se realicen, sus responsables y asegurando la aceptación por parte de las partes involucradas.

La utilización de prototipos es una práctica frecuente para definir y refinar los los requerimientos de manera más precisa, permitiendo a los stakeholders visualizar y experimentar cómo se verá y funcionará la implementación de los requerimientos en el software.

El SRS puede incluir los atributos o funciones externas que el sistema debe tener para interactuar con otros subsistemas o componentes. Esto puede involucrar detalles de diseño que son relevantes para la comprensión del funcionamiento del sistema.

Los detalles particulares de algunos requerimientos específicos son anexados como documentos externos los cuales proporcionan detalles más profundos sobre ciertos aspectos de los requerimientos como es el caso de los H.U, planes de proyecto o de aseguramiento de calidad.

Partes de un SRS

1. Introducción.

- a. Propósito: define el propósito del documento y se especifica a quién va dirigido el documento.
- b. Alcance: da un nombre al sistema futuro. Explica lo que hará y lo que no, da beneficios, objetos y metas.
- c. Referencias: lista completa de todas las referencias de los documentos usados para escribir el SRS.

2. Descripción general:

- a. Perspectiva del producto: si el producto es independiente y autónomo, caso de ser un componente, describe sus dependencias.

- b. Funcionalidad del producto: da un resumen de las funciones que hará.
 - c. Características de los usuarios: describe las características de los usuarios intencionales incluyendo nivel educativo, experiencia y especialización técnica.
 - d. Evolución previsible del sistema: requerimientos implementados a futuro,
3. Requisitos no funcionales.
- a. Requisitos de rendimiento: relacionados a la carga que se espera demorar
 - b. Seguridad: especifica elementos que protegerán al software de accesos, usos maliciosos o sabotajes.
 - c. Portabilidad: atributos del software para que pueda trasladarse o usarse en otras plataformas.
4. Mantenimiento: especifica el mantenimiento necesario del sistema, quién las hará y cuándo.
5. Apéndices: información relevante que no está detallada en los puntos anteriores.
-

Clase 2

Gestión de proyectos

Proceso de planificar, organizar, dirigir y controlar los recursos y actividades necesarias para lograr los objetivos de un proyecto de manera exitosa. Implica una serie de prácticas y metodologías para asegurar que el proyecto se complete dentro de los límites de tiempo, costo y alcance.

Gestión de la configuración del software (GCS)

La GCS es el proceso de identificar los elementos en el sistema, controlando el cambio de estos elementos a lo largo de su ciclo de vida, registrando y reportando el estado de los elementos y las solicitudes de cambio y verificando que los elementos estén completos y que sean los correctos.

Es una actividad de autoprotección que se aplica durante el proceso de software.

Es importante diferenciar el soporte que se hace una vez que se entrega el software al cliente y el GCS.

Los **elementos** de la GCS se pueden dividir en

- Programas.
- Productos de trabajo.
- Datos.

Ya que los elementos de la configuración (ECS) cambian constantemente, se implementa GCS para controlarlos.

El cambio puede deberse a diferentes motivos, los principales serán

1. Nuevas condiciones de negocio o mercado.
2. Nuevas necesidades de las partes interesadas.
3. Reorganización, crecimiento o reducción de la empresa que genera otras prioridades.
4. Restricciones en el presupuesto o calendario.

Como el cambio puede producirse en cualquier momento, la GCS sirve para

- Identificar el cambio.
- Controlar el cambio.
- Garantizar que el cambio se implementará adecuadamente.

- Informar del cambio a todos aquellos que puedan afectarlos.

Proceso

1. Identificación de los elementos en la GCS.
 2. Control de versiones: combina procedimientos y herramientas para gestionar las versiones que se crean a lo largo del proceso de software.
 3. Control de cambios: a lo largo del proyecto los cambios son inevitables y el control es vital para el desarrollo del mismo. Combina los procedimientos humanos y las herramientas adecuadas para proporcionar un mecanismo para el control del cambio. Se evalúa cómo impactará el cambio en hardware, rendimiento, cómo alterará el cambio a la percepción del cliente y si afecta la calidad y fiabilidad.
 4. Auditorías de la configuración: verifica el cambio realizado.
 5. Generación de informes: indicará qué pasó, quién lo hizo, cuándo y qué más se vio afectado. Estos informes son vitales para el éxito del proyecto.
-

Proyecto

Es un esfuerzo temporal que se lleva a cabo para crear un producto, servicio o resultado único y tiene una elaboración gradual al desarrollarse en pasos e ir aumentando mediante incrementos.

Las 4 P de la gestión de proyectos de software

- Personal: elemento más importante. Son quienes realizan el esfuerzo, habrá que identificar a los interesados, sus capacidades y expectativas así como gestionar su participación.
- Producto: el producto de software es intangible y a veces difícil ver progreso.
- Proceso: estructura donde se establece un plan detallado para el desarrollo del software. Es esencial para garantizar una gestión coherente y controlada del proyecto, así como para establecer estándares y procedimientos que guíen el trabajo del equipo.
- Proyecto: los proyectos deben ser planeados y controlados para manejar su complejidad. Implica comprender los factores que contribuyen a su éxito y los riesgos asociados. La gestión adecuada del proyecto abarca la definición de objetivos claros, la asignación de recursos, la monitorización del progreso y la toma de medidas.

Manifestación de una mala gestión de proyectos

- Incumplimiento de plazos; si no se cumple en plazos establecidos, puede tener un impacto en la confianza de los clientes, moral del equipo y la viabilidad general del proyecto.
- Incremento de los costos: si los costos exceden el presupuesto planificado, puede causar dificultades financieras y afectar la rentabilidad.
- Entrega de productos de mala calidad: la baja calidad puede dañar la reputación de la organización y afectar la satisfacción del cliente.

Elementos clave de la gestión de proyectos

- Métricas: el objetivo es controlar y comprender los procesos durante el desarrollo y mantenimiento, además de mejorar los diversos aspectos del proyecto y evaluar su calidad. Brinda además una visión cuantitativa para medir y analizar aspectos del software y el proceso de creación.
- Estimaciones.
- Calendario temporal.
- Organización del personal.

- Análisis de riesgos.
- Seguimiento y control.

El calendario temporal y la organización del personal será dado por la *planificación*.

Planificación

Será una secuencia operativa que especifica

- Qué debe hacerse.
- Con qué recursos.
- En qué orden

Planificación organizativa

El personal que trabaja en una organización de software es el activo más grande ya que representa el capital intelectual.

Una mala administración es uno de los factores principales para el fracaso de los proyectos. Debido a su importancia, se creó el *Modelo de madurez de capacidades del personal (P-CMM)*. Se reconoce que toda organización debe mejorar de manera continua su habilidad de atraer, desarrollar, motivar, organizar y conservar el personal.

Tendrá los siguientes participantes

- Gerentes ejecutivos/ dueños del producto: definen los temas empresariales.
- Gerentes de proyecto (líderes de equipo): planifican, motivan, organizan y controla a los profesionales.
- Profesionales especializados: aportan habilidades técnicas.
- Clientes: especifican los requerimientos.
- Usuarios finales: interactúan con el software.

Líderes de equipo

El equipo de software debe organizarse de manera que maximice las habilidades y capacidades de cada persona.

Las prácticas de líderes ejemplares son:

- Modelar el camino: practicar lo que predicán y demostrar compromiso con el equipo y el proyecto.
- Inspirar y dar visión compartida: es importante motivar a los miembros del equipo, esto implica involucrar a las partes del principio del proceso de establecimiento de metas.
- Desafiar el proceso: tomar la iniciativa de buscar formas innovadoras para mejorar el trabajo y animar a los miembros a experimentar y asumir riesgos.
- Permitir que otros actúen, fomentando habilidades colaborativas del equipo, generando confianza y facilitando relaciones.
- Alentar a celebrar logros individuales y generar espíritu comunitario.

Equipo de software

La mejor estructura de equipo dependerá del número de personas que lo conforman, sus niveles de habilidad y la dificultad del problema en general.

La regla es que no debe tener más de 10 miembros, de todas formas depende mucho del tamaño del proyecto.

Los factores a considerar planeando un equipo será

- Dificultad del problema a resolver.
- Tamaño del programa resultante.

- Tiempo que el equipo permanecerá unido.
- Grado en que se pueden dividir los módulos del problema.
- Calidad y confiabilidad requerida por el sistema a construir.
- Fecha de entrega.
- Grado de sociabilidad requerido en el proyecto.

Sin importar la organización, el objeto es lograr que haya cohesión en el equipo ya que son mucho más productivos y motivados.

Un grupo cohesivo

1. Puede establecer estándares propios de calidad.
2. Se aprende de otros y se apoyan mutuamente.
3. Conocimiento compartido.
4. Alienta la refactorización y la mejora continua.

Si no hay equipo consolidado, puede sufrir toxicidad

1. Atmósfera de trabajo frenética, se evita habiendo acceso a la información, metas y objetivos claros que no se modifican excepto si es necesario.
2. Fricción entre los miembros del equipo, se evita definiendo la responsabilidad de cada uno.
3. Proceso de software mal coordinado, se evita entendiendo lo que se crea.
4. Definición imprecisa de roles.
5. Exposición continua y repetida al fracaso, para evitarlo se deben usar técnicas basadas en retroalimentación y solución de problemas.

La **comunicación grupal** se ve influenciada por el status de los miembros, el tamaño, composición de hombres y mujeres, personalidades y canales de comunicación.

Debe haber mecanismos de comunicación formal e informal.

La formal será con reuniones, escritos y otros medios no interactivos. Mientras que la informal será donde se comparten ideas sobre la marcha.

Muchas organizaciones defienden el desarrollo ágil que conlleva equipos pequeños y motivados, se los denominarán *equipo ágil*.

Hace hincapié en la competencia individual y la colaboración grupal.

Planificación organizativa

Por lo contrario, los proyectos grandes, usarán estructuras de equipos genéricos, los cuales pueden ser

- Descentralizado democrático (DD): no hay un jefe permanente, se nombran coordinadores de tareas a corto plazo, se toman decisiones por consenso y la comunicación es *horizontal*.
- Descentralizado controlado (DC): hay un jefe que coordina tareas específicas y jefes secundarios en subtareas. La resolución de problemas es una actividad del grupo que se reparte en subgrupos. Además, hay comunicación *horizontal y vertical*.
- Centralizado controlado (CC): el jefe del equipo se encarga de la resolución de problemas a alto nivel y la coordinación interna del equipo. La comunicación entre el jefe y miembros es *vertical*.

En conclusión, en una estructura centralizada habrá tareas rápidas y se manejan problemas sencillos mientras que en equipos descentralizados hay más probabilidades de éxito en la resolución de problemas complejos.

La estructura DD es la mejor para problemas difíciles.

Se sugiere que en proyectos muy grandes es mejor seguir estructuras CC o DC para formar subgrupos. Caso en el cual el proyecto dure mucho y el equipo permanezca tiempo juntos, se sugiere DD.

Clase 3

Gestión de riesgos

Un *riesgo* es un evento no deseado que tiene consecuencias negativas.

Es necesario que los gerentes determinen si pueden presentarse eventos no deseados durante el desarrollo y hacer planes para evitarlos o minimizar las consecuencias negativas.

La *deuda técnica* es el término que se usa para describir los costos asociados al aplazamiento de actividades, tales como documentación y refactorización del software.

La deuda técnica no pagada resulta en un producto de mala calidad y genera complejidad innecesaria.

Además implica que los costos de luchar con temas técnicos se puede reducir si se afrontan los problemas al principio, en vez de dejarlos para el final.

Hay 2 estrategias para la gestión de riesgos

- Reactivas: reaccionar ante el problema y "gestionar la crisis".
- Proactivas: tener estrategias de tratamiento.

Riesgos de software

Podrán haber en

- Proyecto.
- Producto.
- Negocio.

Los tipos serán

- Genéricos
 - Conocidos.
 - Predecibles.
 - Impredecibles.
- Específicos
 - Conocidos.
 - Predecibles.
 - impredecibles.

A la hora de gestionar un riesgo

1. Identificación de riesgos: se hará un listado de los potenciales.

Se clasifican en

- Del proyecto: son conocidos.
- Del producto: son predecibles.
- Del negocio: son impredecibles.

Categorización de los riesgos

Riesgo	Repercute en	Descripción
Rotación de personal.	Proyecto.	Personal experimentado abandonará el proyecto antes de que este termine.
Cambio administrativo.	Proyecto.	Habrà un cambio de gestión en la organización con diferentes prioridades.
Indisponibilidad de hardware.	Proyecto.	Hardware que es esencial para el proyecto, no se entrega a tiempo.
Cambio de requerimientos.	Proyecto y producto.	Especificaciones de interfaces esenciales no están disponibles a tiempo.
Subestimación del tamaño.	Proyecto y producto.	Se subestimó el tamaño del sistema.
Bajo rendimiento de las herramientas CASE.	Producto.	Las herramientas CASE, que apoyan el proyecto, no se desempeñan como se anticipaba.
Cambio tecnológico.	Empresa.	La tecnología subyacente sobre la cual se construye el sistema se sustituye con nueva tecnología.
Competencia de productos.	Empresa.	Un producto competitivo se comercializa antes de que el sistema esté completo.

2. Análisis de riesgos: listado de priorización de riesgos. Cada riesgo es identificado con su probabilidad e impacto para construir la tabla de riesgos.

Se debe establecer una escala que refleje la probabilidad observada de un riesgo.

- Bastante improbable: < 10%.
- Improbable: 10-25%.
- Moderado: 25-50%.
- Probable: 50-75%.
- Bastante probable: > 75%.

Se estima el impacto en el proyecto:

1. Catastrófico: cancelación del proyecto.
2. Serio: reducción de rendimiento, retrasos en la entrega, excesos importantes en costo.
3. Tolerable: reducciones mínimas de rendimiento, posibles retrasos, exceso en costo.
4. Insignificante: incidencia mínima en el desarrollo.

Boehm recomienda

- Identificar y supervisar los 10 riesgos más altos.
- El número exacto de riesgos debe depender del proyecto.
- Los riesgos que queden encima de la línea serán los que se les preste atención.
- Los que queden debajo de la línea serán reevaluados y tendrán una prioridad de segundo orden.

Un factor de riesgo que tenga **gran impacto** pero **poca probabilidad** de que ocurra, no debería absorber un tiempo significativo.

Los riesgos de **gran impacto** con una **probabilidad de moderada a alta** y los riesgos de **poco impacto** pero con **gran probabilidad** deberían tomarse en cuenta.

3. Planeación de riesgos: anulación de riesgos y planes de contingencia.

Se consideran cada uno de los riesgos por encima de la línea de corte y se determina una estrategia a seguir.

- Evitar el riesgo: siguiendo esta estrategia, el sistema se diseña de modo que no pueda ocurrir el evento.
- Minimizar el riesgo: siguiendo esta estrategia, la probabilidad que el riesgo se presente se reduce.
- Plan de contingencia: se acepta la ocurrencia del riesgo y es tratado de manera de minimizar las consecuencias.

Para la toma de decisión acerca del tratamiento de los riesgos, se debe tener en cuenta el costo de la aplicación de las estrategias.

$INFLUENCIA = (Exposición\ antes - exposición\ después) / costo\ de\ reducción.$

Exposición: probabilidad que ocurra por el costo del proyecto si sucede el riesgo.

Para que se justifiquen las acciones de reducción del riesgo, el valor de influencia debe ser alto.

4. Supervisión de riesgos: valoración de riesgos.

- Evaluar si ha cambiado la probabilidad de cada riesgo.
- Evaluar la efectividad de las estrategias propuestas.
- Detectar la ocurrencia de un riesgo fue previsto.
- Asegurar que se están cumpliendo los pasos definidos para cada riesgo.
- Recopilar información para el futuro.
- Determinar si existen nuevos riesgos.
- Reevaluar periódicamente los riesgos.
- Los riesgos deben monitorizarse comúnmente en todas las etapas del proyecto. En cada revisión administrativa, es necesario reflexionar y estudiar cada uno de los riesgos clave por separado.
- También hay que decidir si es más o menos probablemente que surja el riesgo, y si cambiaron la gravedad y las consecuencias del riesgo.

Clase 4

Interface de usuario

Se basa en diseño de computadoras, aplicaciones, máquinas, dispositivos de comunicación móvil, aplicaciones web y sitios.

Es una actividad multidisciplinar que involucra a varias ramas del diseño y el conocimiento como el diseño gráfico, industrial, web y de software y está implicado en un amplio rango de proyectos desde sistemas de computadoras hasta aviones comerciales.

El objetivo es mantener la interacción con los destinatarios de una forma más atractiva, centrando el diseño en ellos.

El diseño gráfico y diseño industrial basan sus conocimientos para que los usuarios aprendan lo más rápido posible el funcionamiento del software.

Las herramientas principales que utilizan son recursos como la gráfica, lo estético y la simbología, sin afectar el funcionamiento técnico eficiente.

Conceptos iniciales

El diseño es el medio de comunicación entre el hombre y la máquina por lo que usando un conjunto de principios se crea un formato de pantalla.

Será necesario estudiar las preferencias de las personas para producir tecnología que se adapte a los seres humanos.

Una interfaz difícil de utilizar provoca que los usuarios cometan errores o incluso que se rehúsen a usar el sistema.

Surge la diversidad, lo que se refiere a que las personas pueden tener estilos diferentes de percepción, comprensión y trabajo. Por lo que la interfaz debe contribuir a que el usuario consiga un rápido acceso al contenido sin pérdida de comprensión mientras se desplaza a través de la información.

La variedad de tecnologías que deben adaptarse al usuario serán: hipertexto, sonido, presentaciones tridimensionales, video, realidad virtual, etc. Como también configuraciones de hardware como teclado, mouse, dispositivos de presentación, lápices, reconocimiento de voz, etc. Además de PC, equipos específicos, celulares, etc.

6 principios para el diseño de la interfaz del usuario

- Familiaridad del usuario: utilizar términos y conceptos que se toman de la experiencia de las personas que más utilizan el sistema.
- Consistencia: siempre que sea posible, la interfaz debe ser consistente en el sentido de que las operaciones comparables se activan de la misma forma.
- Mínima sorpresa: el comportamiento del sistema no debe provocar sorpresa a los usuarios.
- Recuperabilidad: la interfaz debe incluir mecanismos para permitir a los usuarios recuperarse de los errores.
- Guía del usuario: cuando los errores ocurren, la interfaz debe proveer retroalimentación significativa y características de ayuda sensible al contexto.
- Diversidad de usuarios: la interfaz debe proveer características de interacción apropiada para los diferentes tipos de usuarios.

Proceso

El proceso de análisis y diseño de las UI es iterativo.

- Validación de la interfaz: la capacidad de la interfaz para implementar todas las tareas del usuario. La interfaz es fácil de usar y aprender y la aceptación deberá de ser por parte del usuario.
- Análisis y modelado de la interfaz: entender el problema, a los usuarios, las habilidades, el negocio, etc.
- Construcción de la interfaz: creación de un prototipo que permite evaluar los escenarios de uso.
- Diseño de la interfaz: objetos, acciones y representaciones que permitan efectuar las tareas.

1. Analizar y comprender las actividades del usuario.
2. Realizar el diseño del prototipo en papel.
3. Evaluar el diseño con los usuarios finales.
4. Realizar el diseño dinámico del prototipo.
5. Evaluar el diseño con los usuarios finales.
6. Implementar la interfaz de usuarios definitiva.

Diseño de experiencias de usuario (UX)

UX es un conjunto de métodos aplicados al proceso de diseño que busca satisfacer las necesidades del cliente y proporciona buena experiencia a los usuarios destinatarios.

Tipos de diseño UX

- Diseño de interacción: es la interacción entre un usuario y producto, donde el objetivo es que sea agradable para el usuario.
- Diseño visual: sirve para comprobar cómo se ve y cómo se siente navegar por una aplicación; además de la eficiencia, nivel de entretenimiento. Incluye equilibrio, espacio, contraste, color, forma y tamaño.
- Investigación del usuario: determina qué quieren y necesitan los usuarios.
- Arquitectura de la información: los diseñadores lo usarán para estructurar y etiquetar el contenido de forma que los usuarios puedan encontrar la información fácilmente.

Proceso del diseño de la UX

1. Investigación: obtiene toda la información posible del proyecto y del producto a diseñar. Se toma información de diferentes fuentes como encuestas, información de ventas, mercadotecnia, charlas de apoyo al usuario. Del usuario se relevará la siguiente información: franja etaria, etnia, género, ingresos, idioma, nivel de estudios, localización, ocupación, religión.
2. Organización: organiza toda la información para convertirla en un producto.
3. Diseño: propone el diseño del producto a partir de la organización.
4. Prueba: comprueba el diseño puesto en el producto.

Aspectos de diseño de interfaz

Reglas básicas del diseño

- Dar control al usuario: el usuario busca un sistema que reaccione a sus necesidades y lo ayude a hacer sus tareas. Habrá que definir modos de interacción de forma que el usuario no realice acciones innecesarias, dar una interacción flexible, incluir opciones de deshacer e interrumpir, ocultar elementos técnicos internos, diseñar interacción directa con objetos que aparecen en pantalla.
- Reducir la carga de memoria: definir valores por defecto que tengan significado, definir accesos intuitivos, desglosar información progresivamente.
- Lograr una interfaz consistente: el usuario debe saber de dónde viene y hacia dónde va, mantener consistencia en toda la familia de aplicaciones, usar mismas reglas de diseño para las mismas interacciones, mantener modelos prácticos.

Reglas básicas del diseño

- Factores humanos: percepción visual/ auditiva/ táctil.
- Memoria humana.
- Razonamiento.
- Capacitación.
- Comportamiento.
- Diversidad de usuarios.

Usabilidad

La usabilidad se obtiene cuando la arquitectura de la interfaz se ajusta a las necesidades de las personas que la emplearán.

Es debatible la definición de usabilidad, otra según Donahue es que es *una medida de cuán bien un sistema facilita el aprendizaje, ayuda a quienes lo emplean a recordar lo aprendido, reduce la probabilidad de cometer errores y les permite ser eficientes.*

Para determinar si la usabilidad está presente en un sistema será preguntando las siguientes cosas

- Si el sistema es utilizable sin ayuda o enseñanza.
- Si las reglas de interacción ayudan al usuario a trabajar con eficiencia.
- Si los mecanismos de interacción se hacen más flexibles a medida que los usuarios conocen mpas.
- Si está adaptado el sistema al ambiente físico y social donde se usará.
- Si la interfaz está estructurada de manera lógica y consistente.

Principios de usabilidad- Jacob Nielsen

- Dialogo simple y natural: forma en que la interacción con el usuario puede llevarse a cabo. Realizar una escritura correcta, sin errores de tipeo, no mezclar información importante con la irrelevante, distribuir la información adecuadamente, evitar uso excesivo de mayúsculas y abreviaturas.
- Lenguaje del usuario: emplear en el sistema un lenguaje familiar para el usuario. Evitar palabras técnicas, diseñar correctamente las entradas de datos y emplear un agrado adecuado de información.
- Minimizar el uso de la memoria del usuario: evitar que el usuario esfuerce su memoria para interactuar con el sistema.
- Consistencia: que no existan ambigüedades en el aspecto visual ni tecnológico en el diálogo o en el comportamiento del sistema.
- Feedback: respuesta gráfica o textual en la pantalla frente a una acción del usuario. El sistema debe mantener al usuario informado de lo que está sucediendo.
- Salidas evidentes: que el usuario tenga a su alcance de forma identificable y accesible una opción de salida.
- Mensajes de error: información que brinda el sistema ante la presencia de un error y de qué forma se ayuda al sistema para que salga de la situación en la que está.
- Prevención de errores: evitar que el usuario llegue a una instancia de error.
- Atajos: la interfaz debería proveer de alternativas de manejo para que resulte cómodo y amigable tanto para usuarios novatos como para usuarios experimentados.
- Ayudas: componentes de asistencia para el usuario. Un mal diseño de las ayudas puede llegar a entorpecer y dificultar la usabilidad.

Existen ciertos principios de diseño que enuncian el diálogo correcto que debe proveer una interfaz de usuario. Estos principios fueron desarrollados por Nielsen y son usados para el diseño de interfaces y, como métricas de evaluación de interfaces ya desarrolladas.

Estilos de interfaces

- Interfaz de comandos: interfaz más elemental donde solo se interactúa con texto, usualmente desde una línea de comandos.
Es poderoso y flexible, hay una administración pobre de errores y es difícil de aprender.
- Interfaz de selección de menú: se presentan un conjunto de opciones que pueden ser seleccionadas por el usuario y solo se interactúa con los caracteres indicados. Evita errores y es lento para usuarios experimentados.
- GUI, interfaz gráfica de usuarios: se caracteriza por la utilización de todo tipo de recursos visuales para la representación e interacción con el usuario. Son relativamente fáciles de aprender y utilizar, hay pantallas

múltiples y se tiene acceso inmediato a cualquier punto de la pantalla.

- Interfaz de relleno de formularios: introducción de datos sencilla en los campos de un formulario, es fácil de aprender pero ocupa mucho espacio en pantalla.
 - Manipulación directa: interacción directa con los objetos de la pantalla.
 - Interfaz de reconocimiento de voz.
 - Interfaz inteligente.
-

Clase 5

Planificación temporal

Actividad que distribuye el esfuerzo estimado a lo largo de la duración del proyecto. Es importante su precisión para evitar clientes insatisfechos, costos adicionales, etc.

Calendarización del proyecto: acción que distribuye el esfuerzo estimado a través de la duración planificada del proyecto, asignando el esfuerzo a tareas específicas del desarrollo de software.

Existen dos casos

- Fecha final fijada por cliente: se obliga a distribuir el esfuerzo dentro del plazo previsto.
- Fecha final fijada por desarrolladores: el esfuerzo se distribuye para conseguir un uso óptimo de los recursos y se define una fecha de fin luego del riguroso análisis.

Se compone por

1. Tarea: secuencia de acciones a hacer en un plazo determinado. Podrá describirse con cuatro parámetros
 - a. Precursor: evento o conjunto de eventos que deben ocurrir antes de que la actividad pueda comenzar.
 - b. Duración: tiempo necesario para completarlo.
 - c. Fecha de entrega: para la cual debe estar completada.
 - d. Resultado: componente listo.
2. Tarea crítica: aquella cuyo retraso genera uno en todo el proyecto.
3. Hito: algo que se espera que esté hecho para alguna fecha.

Red de tareas

Representación gráfica del flujo de las tareas desde el inicio hasta el fin de un proyecto. Algunas tareas se pueden realizar en paralelo.

El conjunto de tareas varía dependiendo del tipo de proyecto y su grado de rigor.

Los factores que influyen en el conjunto de tareas a elegir son

- Tamaño del proyecto.
- Número de usuarios potenciales.
- Criticidad del proyecto.
- Estabilidad de los requerimientos.
- Facilidad de comunicación con el cliente/ usuario.
- Madurez de la tecnología aplicable.
- Restricciones.

Método de planificación temporal

- PERT (Program Evolution & Review Technique) fue creado para proyectos del programa de defensa de EE.UU el cual se usa para controlar la ejecución de proyectos con gran número de actividades que implican investigación, desarrollo y pruebas.

Hay una red de tareas con fechas tempranas, tardías, camino crítico y probabilístico.

- CPM (Critical Path Method) fue desarrollado por dos empresas estadounidenses el cual se usa en proyectos en los que hay poca incertidumbre en las estimaciones, el tiempo de inicio es temprano y tardío y es determinístico.
-

Clase 6

Elementos clave de la gestión de proyectos

- Métricas.
- Estimaciones.
- Calendario temporal.
- Organización del personal.
- Análisis de riesgos.
- Seguimiento y control.

Las **métricas** pueden ser utilizadas para que los profesionales e investigadores puedan tomar las mejores decisiones y además como un medio para asegurar la calidad en procesos, proyectos de software o productos.

Las métricas del **proyecto** se usarán para

- Ajustes en el calendario y evitar demoras.
- Valorar el estado de un proyecto en marcha.
- Rastrear riesgos.
- Descubrir áreas de problemas.
- Ajustar flujo de trabajo/ tareas.
- Evaluar habilidad del equipo.

Las métricas del **proceso** se usarán para

- Sentido común y sensibilidad organizacional.
- Retroalimentación.
- No usar métricas para valorar a los individuos.
- Establecer metas y métricas claras.
- No excluir métricas.

La única forma de mejorar un proceso es medir atributos específicos de este, desarrollar un conjunto de métricas en base a estos atributos y luego usarlos para lograr indicadores que llevarán a la mejora del proceso. Por ejemplo: errores descubiertos antes de liberar el software, productos de trabajo entregados, esfuerzo humano invertido.

Métricas del producto

Son medidas cuantitativas usadas para evaluar y cuantificar diversos aspectos del software que proporcionan información objetiva sobre características y cualidades.

- Dinámicas: realizadas durante la ejecución del programa las cuales evalúan la eficiencia y fiabilidad del software.

- Estáticas: realizadas en función de las representaciones del sistema, evalúan la complejidad, comprensibilidad y mantenibilidad del software.

Métricas postmortem

Referido al análisis y evaluación de métricas después de que un proyecto haya sido completado. Se usan para analizar retrospectivamente el desempeño, éxito y resultados de un proyecto una vez que finalizó.

- Línea base: da una línea base para métricas futuras.
- Mantenimiento: facilita el mantenimiento al conocer la complejidad lógica, tamaño y flujo.
- Reingeniería: contribuye a los procesos de reingeniería al proporcionar información de rendimiento y estructura.

Métricas del producto

Estimaciones

Técnicas que permiten asignar un valor aproximado a una cantidad, como tiempo, recursos, costos o cualquier otro aspecto. Se usan para proporcionar una idea general y rápida de lo que se espera.

Las estimaciones se usan cuando no es posible aplicar métricas por falta de datos o la naturaleza de la tarea. Es útil cuando se necesitan ideas rápidas.

Las estimaciones se aplican a diversos aspectos como recursos, proyectos, costos, tiempo y más.

Se debe comprender que las estimaciones pueden ser precisas o no y algunos factores pueden alterarlas por lo que será necesario tener experiencia para que resulten efectivas.

Los factores que pueden influir será la complejidad del proyecto, estructuración u otros aspectos relacionados.

Las métricas son datos concretos con medidas seguras y precisas mientras que las estimaciones son valores aproximados y más subjetivos y se usa en situaciones donde la información precisa es difícil de obtener.

Estimación de recursos

- Recursos humanos
 - Cantidad: determinar cuántas personas serán necesarias para el proyecto.
 - Habilidades: identificar las habilidades y competencias específicas requeridas para cada rol en el proyecto.
 - Duración: estimar el tiempo que cada miembro necesitará para completar sus tareas.
 - Ubicación: definir dónde está físicamente el equipo o si habrá algún integrante remotamente.
- Entorno
 - Herramientas de software: identificar y estimar el uso de las herramientas necesarias.
 - Hardware: prever el hardware necesario como servidores o dispositivos.
 - Recursos de red: evaluar la infraestructura de red necesaria.
- Software reutilizable
 - Componentes COTS: evaluar adquisición de componentes de software que proveen una funcionalidad específica y están disponibles en el mercado para ser adquiridos e integrados.
 - Componentes nuevos: identificar cualquier nuevo componente que necesite ser desarrollado para el proyecto.
 - Componentes de experiencia completa: componentes que ya se hayan usado o desarrollado en proyectos anteriores.

- Componentes de experiencia parcial: componentes desarrollados parcialmente que requieran ajustes y pruebas adicionales.

Estimación de costos

- Costos de esfuerzos: relacionados con los RR.HH, específicamente los salarios y remuneraciones del equipo de desarrollo.
- Hardware y software: costos asociados con la infraestructura tecnológica. Abarca el hardware así como el software usado como herramientas de gestión del proyecto y otras soluciones.
- Viajes: costos relacionados a la movilidad del equipo.

Estimación de costos- Fijación de precio y relación precio-costo

El costo se refiere a los gastos y recursos necesarios para completar un proyecto mientras que el precio es la cantidad que se cobra al cliente por algo entregado.

- Oportunidades de mercado: evaluar la demanda y la competencia para determinar un precio que sea atractivo y competitivo.
- Incertidumbre en estimaciones de costos: reconocer que pueden ser inexactas y ajustar el precio para mitigar riesgos financieros.
- Términos contractuales: considerar acuerdos y compromisos asumidos con los clientes puede influir en la fijación de precios.
- Volatilidad de requerimientos: anticipar cambios y cómo pueden afectar en los costos.
- Salud financiera: evaluar la situación financiera de la empresa y determinar si cubre costos y permite rentabilidad.

En conclusión, la fijación de precios implica un equilibrio entre los costos, rentabilidad y competitividad en el mercado. La fijación debe considerar los intereses como las necesidades y expectativas del cliente.

Técnicas de estimación

- Juicio experto: técnica usada para evaluar insumos, detalles técnicos y aspectos de gestión de proyectos. Proporciona una perspectiva externa beneficiosa, especialmente en proyectos de alto impacto. Además de su aplicación principal, el juicio de expertos también es útil en situaciones clave:
 - Validación: en proyectos de alto impacto cuando hay mucho en juego y es necesaria la validación para avanzar.
 - Pre-planificación: antes de la planificación completa del proyecto.
 - Precio-costo: para la gestión de costos y determinar precios de productos.
 - Riesgos: en la gestión de riesgos.
- Técnica Delphi: un grupo de expertos estima anónimamente. Cada ronda recompila estimaciones individuales y se proporciona sin revelar la identidad. El proceso se repite sucesivamente en rondas hasta que haya un consenso.
- División de trabajo jerárquica: implica la consulta a personas en distintos niveles jerárquicos.
- Planning poker: técnica colaborativa usada en la estimación de esfuerzo para las H.U durante un sprint de desarrollo.

Los pasos a seguir serán

- Reunión del equipo: todos los miembros se reúnen y poseen cartas numeradas basadas en la serie de Fibonacci.

- Lectura de historias: el P.O presenta una H.U al equipo.
 - Selección de cartas: cada miembro elige una carta con el número que considera representar el esfuerzo requerido para completar la historia.
 - Revelación y comparación: las cartas se revelan.
 - Comparación y justificación: se comparan las cartas y se brindan justificaciones para las elecciones hechas. Si hay consenso se asigna el valor de la carta a la historia.
-

Modelos empíricos de estimación

Se aplican a proyectos de gran envergadura debido a su complejidad. Estos modelos son fórmulas desarrolladas a partir de datos recopilados para prever los costos o esfuerzo necesario.

COCOMO II (Constructive cost model)

Su objetivo es estimar el esfuerzo, costo y tiempo para desarrollar un sistema. Se basará en la recopilación y análisis de datos históricos de proyectos previos para establecer relaciones entre diversos factores.

Vincula fórmulas sobre el tamaño del sistema, del producto, del proyecto y del entorno para estimar el esfuerzo de horas por persona, costo y duración.

Es una evolución de COCOMO I que se centraba en las líneas de código y el tamaño del sistema y que su evolución adopta metodologías ágiles y que reconoce la dificultad de medir líneas de código por lo que se enfoca en prototipos, desarrollo espiral o basado en componentes.

Clase 7

Diseño de software

Etapla fundamental en el cual se crea una representación detallada y estructurada de cómo se construirá el software final, teniendo en cuenta los requisitos del sistema, necesidades y consideraciones técnicas.

Es esencial porque representa la primera actividad técnica en el proceso de desarrollo. Se establece la calidad, ya que generan múltiples representaciones de desarrollo y que se evalúan en términos de esta.

Un diseño deficiente o la ausencia de diseño conlleva el riesgo de crear un sistema inestable que podría fallar incluso con cambios mínimos en su implementación.

Áreas

- Diseño de datos: transforma el modelo del dominio, derivado del análisis, en estructuras concretas de datos, objetos de datos y relaciones. Este diseño define cómo se almacenarán y gestionarán los datos.
- Diseño arquitectónico: relación entre los componentes estructurales y selecciona los estilos arquitectónicos, patrones de diseño y otras decisiones clave para la estructura general del sistema.
- Diseño a nivel de componentes: convierte los elementos arquitectónicos en una descripción detallada de los componentes de software y sus funcionalidades. Se basa en modelos de clases, flujos y comportamientos.
- Diseño de interfaz: define cómo se comunicará el software tanto internamente como con sistemas externos y personas.

Características para su evolución

- Deberá implementar todos los requerimientos explícitos del modelo de requerimientos e incorporar todos los implícitos que desee el cliente.
- Ser una guía legible y comprensible para aquellos que generan código y para aquellos que dan soporte al software.
- Deberá proporcionar una visión completa del software.

Criterios técnicos para un buen diseño

- Presentar una estructura arquitectónica que se haya creado mediante patrones de diseño reconocibles, se implemente evolutivamente y cada componente tenga buen diseño.
- Ser modular.
- Tener distintas representaciones.
- Conducir a interfaces que reduzcan la complejidad de las conexiones entre módulos y con el entorno externo.
- Derivarse mediante un método repetitivo y controlado por la información obtenida durante el análisis de los resultados del software.
- Representarse por medio de una notación que comunique de manera eficaz su significado.

Evolución del diseño de software

La evolución del diseño de software a lo largo de más de seis décadas fue un proceso constante que abarcó diversas metodologías y enfoques. Desde programas modulares y refinación de estructuras de software hasta los enfoques orientados a objetos, a modelos o los basados en pruebas, cada etapas trajo innovaciones de cómo abordar el diseño de sistemas y aplicaciones.

A lo largo de estas diferentes fases, se observaron características comunes que se aplican a diversos enfoques de diseño.

1. Los enfoques de diseño con un mecanismo para traducir el modelo de requerimientos en una representación de diseño.
2. Son una notación para presentar los componentes funcionales y sus interfaces.
3. Se emplean heurísticas para refinamiento.
4. Son lineamientos para evaluar la calidad.

Sin importar el tipo de diseño, será necesario aplicar algunos conceptos básicos.

- Criterios para dividir el software en componentes individuales.
- Cómo se extraen los detalles de la función o la estructura de datos para la representación conceptual del software.
- Criterios que definen la calidad técnica de un diseño.

Conceptos de diseño

- Abstracción: permite enfocarnos en problemas a un nivel general sin importar algunos detalles de bajo nivel. La abstracción podrá ser
 - Procedimental: secuencia de instrucciones que tienen una funcionalidad específica.
 - De datos: colección de datos que definen un objeto real.
- Arquitectura: estructura general del software y las formas en la que la estructura da una integridad conceptual para un sistema.

- Patrones: describen una estructura de diseño que resuelve un problema particular dentro de un contexto específico.
- Modularidad: el software se divide en componentes nombrados y abordados por separado llamado módulos, se integrarán para satisfacer requisitos del problema.
- Ocultamiento de información: la información que está dentro de un módulo es inaccesible a otros que no la necesiten.
- Independencia funcional: sumatoria de modularidad, abstracción y ocultamiento de la información.

Busca lograr la alta cohesión y el bajo acoplamiento.

Se entiende por cohesión a la medida de relación que hay entre sentencias de un mismo módulo y acoplamiento es la interconexión entre dos módulos.

El nivel de acoplamiento puede ser bajo, moderado o alto.

- Refinamiento: se refina de manera sucesiva. La abstracción y el refinamiento son conceptos complementarios porque la abstracción permite especificar procedimientos y datos sin dar detalles y el refinamiento ayudará a revelar los detalles de grado menor.
- Refabricación: reorganización que simplifica el diseño de un componente si cambia su función o comportamiento.

Principios del modelado de diseño

- El diseño debe poder rastrearse hasta el modelo de requerimientos.
- Considerar siempre la arquitectura del sistema que se va a crear.
- El diseño de los datos es tan importante como el diseño de funciones.
- El diseño de interfaz debe ajustarse a las necesidades del usuario, haciendo énfasis en la facilidad de uso.
- El diseño a nivel de componentes debe ser funcionalmente independiente.
- Los componentes deben tener bajo acoplamiento.
- El diseño debe desarrollarse de manera iterativa.
- La creación de un modelo de diseño no impide metodología ágil.

Clase 8

Diseño arquitectónico

Define la relación entre los elementos estructurales para lograr los requisitos del sistema. Identifica los subsistemas dentro del sistema y establece un marco de control y comunicación entre ellos.

Los grandes sistemas se dividen en subsistemas que proporcionan algún conjunto de servicios relacionados.

La arquitectura afecta directamente a los requerimientos *no funcionales* los cuales serán

- Críticos: rendimiento, protección, seguridad, disponibilidad, mantenibilidad.
- Rendimiento: se deben agrupar las operaciones críticas en un grupo reducido de subsistemas.
- Seguridad: se debe usar una arquitectura en capas, protegiendo los recursos más críticos en capas más internas.
- Protección: la arquitectura deberá diseñarse para que las operaciones relacionadas con la protección se localicen en un único subsistema para reducir costos y problemas de validación.
- Disponibilidad: la arquitectura se deberá diseñar con componentes redundantes para que sea posible el reemplazo sin detener el sistema.

- **Mantenibilidad:** la arquitectura debe diseñarse con componentes autocontenidos de grano fino que puedan modificarse con facilidad.

Características del diseño arquitectónico

1. **Organización del sistema:** estrategia utilizada para estructurar el sistema. Es la manera en que se dividen y conectan los subsistemas para lograr los objetivos y requisitos del proyecto. Una estructura organizada eficazmente facilita la comunicación, el control y la interacción entre los diferentes elementos del sistema. Los subsistemas deben intercambiar información de forma efectiva. Esto se puede lograr de diferentes maneras, entre ellas
 - **Repositorio:** la mayoría de los sistemas que usan grandes cantidades de datos se organizan alrededor de una base de datos compartida. Los datos son generados por un subsistema y utilizados por otros.
 La ventaja es que es eficiente para compartir gran cantidad de datos, los sistemas productores de datos no deben saber cómo se utilizan, las actividades de backup, protección y control de acceso está centralizado.
 Las desventajas es que los subsistemas deben estar acordes a los modelos de datos del repositorio y la evolución suele ser difícil a medida que se genera gran volumen de información además de que los subsistemas pueden tener distintos requerimientos de protección o políticas además de ser difícil distribuir el repositorio a varias máquinas.
 - **Cliente-servidor:** modelo donde el sistema se organiza como un conjunto de servicios y servidores asociados, más unos clientes que usan los servicios.
 Estará compuesto por un conjunto de servidores que dan servicios, clientes que llaman a los servicios y una red que permite a los clientes acceder a los servicios.
 - **Capas:** el sistema se organiza en capas, donde cada una presenta un conjunto de servicios a sus capas adyacentes. Lo bueno es que soporta el desarrollo incremental, es portable y resistente a cambios, una capa puede ser reemplazada siempre que se mantenga la interfaz y termine generar sistemas multiplataforma ya que solamente las capas más internas son dependientes de la plataforma.
 Lo malo es que resulta difícil de estructurar, las capas internas proporcionan servicios que son requeridos por todos los niveles, hay que atravesar muchas capas.
 - **Combinaciones entre ellos**
2. **Descomposición modular:** estrategia para dividir los subsistemas en módulos más pequeños y manejables. A estos módulos se les puede aplicar los patrones y modelos clásicos, pero también permite utilizar otros estilos alternativos.
 Un sistema es independiente cuando no depende de los servicios de otros para funcionar.
 La descomposición modular posee dos estrategias: *orientada a flujo de funciones* y *orientada a objetos*.
 En el flujo de funciones se ingresan datos y transforman en salida mientras que en la orientación a objetos hay un conjunto de objetos que se comunican.
 - **Subsistema:** sistema cuyo funcionamiento no depende de los servicios proporcionados por otros.
 - **Módulo:** es un componente de un subsistema que proporciona uno o más servicios a otros módulos.
3. **Modelos de control:** en un sistema, los subsistemas están controlados para que sus servicios se entreguen en el lugar correcto en el momento preciso.
 - **Control centralizado:** un subsistema tiene la responsabilidad de iniciar y detener otro subsistema. Un subsistema se diseña como controlador y tiene la responsabilidad de gestionar la ejecución de otros subsistemas, la ejecución puede ser secuencial o en paralelo.
 - **Control basado en eventos:** se rigen por eventos generados externamente al proceso. Se podrá clasificar en
 - **Modelos de transmisión:** se transmite a todos los subsistemas y cualquiera tendrá la capacidad de tomarlo y atenderlo.

- Modelo dirigido por interrupciones: se usa en sistemas de tiempo real donde las interrupciones externas son detectadas por un manejador de interrupciones y se envía a algún componente para su procesamiento.

4. Arquitectura de los sistemas distribuidos: un sistema distribuido es un sistema en el que el procesamiento de información se distribuye sobre varias computadoras.

Tipos genéricos de sistemas distribuidos

- Cliente-servidor: el sistema se diseña como un conjunto de servicios proporcionados por servidores y un conjunto de clientes que acceden a estos servicios a través de una red de comunicación.

Los clientes y servidores son procesos diferentes desde la óptica de sistemas distribuidos con concurrencia. Operan en equipos distintos, lo que asegura que los procesos de cliente y servidor se ejecuten independientemente. Los servidores pueden atender a varios clientes a la vez lo que mejora la respuesta del sistema.

Los clientes no se conocen entre sí y operan independientemente, operando según sus necesidades

- Componentes distribuidos: esta arquitectura se basa en el diseño de sistemas como conjuntos de componentes u objetos que ofrecen una interfaz para acceder a un conjunto de servicios que proporcionan. A diferencia de cliente-servidor, aquí no existe una distribución rígida sino que todos los componentes son considerados como objetos distribuidos.

Los componentes pueden estar en diferentes máquinas y equipos conectados a través de una red y se comunicarán para colaborar entre sí.

Características de los sistemas distribuidos

1. Compartir recursos: un sistema distribuido permite compartir recursos.
2. Apertura: son sistemas abiertos y se diseñan con protocolos estándar para simplificar la combinación de los recursos.
3. Concurrencia: varios procesos pueden operar al mismo tiempo sobre diferentes computadoras.
4. Escalabilidad: la capacidad puede incrementarse añadiendo nuevos recursos para cubrir nuevas demandas.
5. Tolerancia a fallos: la disponibilidad de varias computadoras y el potencial para reproducir información hace que los sistemas distribuidos sean más tolerantes a fallas de hardware y software.

Desventajas de los sistemas distribuidos

1. Complejidad: son más complejos que los centralizados porque además del procesamiento se debe tener en cuenta la comunicación y sincronización.
2. Seguridad: se accede al sistema desde varias computadoras generando tráfico en la red que puede ser intervenido.
3. Manejabilidad: las computadoras del sistema pueden ser de diferentes tipos o S.O lo que genera más dificultades en la gestión y mantenimiento.
4. Impredecibilidad: la respuesta depende del estado de red y la carga del sistema, lo que hará que el tiempo de respuesta varíe.

Arquitectura de los sistemas distribuidos

- Multiprocesador: el sistema de software está formado por varios procesos que pueden o no ejecutarse en procesadores diferentes. La asignación de los procesadores pueden ser predeterminada o mediante un dispatcher.

Es común en sistemas grandes de tiempo real que recolectan información, toman decisiones y envían señales para modificar el entorno.

- Cliente-servidor: una aplicación se modela como un conjunto de servicios proporcionado por los servidores y un conjunto de clientes que los usan. Los clientes y servidores son procesos diferentes, los servidores pueden atender varios clientes y brindar varios servicios, además los clientes no se conocen entre sí.

Esta arquitectura se puede clasificar en niveles

- Dos niveles: donde hay dos tipos de clientes. *Cliente ligero*, donde el procesamiento y gestión de datos se lleva a cabo en el servidor y el *cliente pesado* donde el cliente implementa la lógica de la aplicación y el servidor solo gestiona los datos.
- Multinivel: la presentación, el procesamiento y la gestión de datos son procesos lógicamente separados y se pueden ejecutar en procesadores diferentes.
- Computación distribuida inter-organizacional: una organización tiene varios servidores y reparte su carga computacional entre ellos. Extiende este concepto a varias organizaciones, podrá clasificarse en:
 - Peer to peer: son sistemas descentralizados en los que el cálculo puede llevarse a cabo en cualquier nodo de la red. Se diseñan para aprovechar la ventaja de la potencia computacional y el almacenamiento a través de una red.

Pueden usar una arquitectura

- Descentralizada, donde cada nodo rutea los paquetes a sus vecinos hasta encontrar el destino.
- Semi-centralizada, donde un servidor ayuda a conectarse a los nodos o coordinar resultados.
- Orientada a servicios: un *servicio* es una representación de un recurso computacional o de información que puede ser utilizado por otros programa.

Un servicio es independiente de la aplicación que lo usa y puede ser usado por varias organizaciones.

Una aplicación podrá construirse enlazando varios servicios y resultan ser débilmente acopladas.

Un proveedor de servicios oferta servicios definiendo su interfaz y su funcionalidad y para que sea externo, el proveedor lo publica con información del mismo. Un solicitante enlaza este servicio a su aplicación para invocarlo y procesar el resultado del mismo.

Codificación

Una vez establecido el diseño, se deben escribir los programas que implementen dicho diseño.

Esto puede resultar una tarea compleja por distintos motivos

- Los diseñadores pueden no haber tenido en cuenta las particularidades de la plataforma y el ambiente de programación.
- Las estructuras e interrelaciones que son fáciles de describir mediante diagramas, no siempre resultan sencillas de escribir en código.
- Es indispensable escribir el código de forma que resulte comprensible para otras personas.
- Se deben sacar beneficios de las características de diseño creando código reutilizable.

Resulta útil conservar la **calidad del diseño en la codificación**

- Localización de entrada y salida: es deseable localizarlas en componentes separados del resto del código ya que generalmente son más difíciles de probar.
- Inclusión de pseudocódigo: es útil avanzar el diseño, realizando un pseudocódigo para adaptar el diseño al lenguaje elegido.
- Revisión y reescritura: es recomendable realizar un borrador, revisarlo y reescribirlo tantas veces como sea necesario.
- Reutilización: hay dos clases

- Productiva: se crean componentes destinados a ser reutilizados por otra aplicación.
- Consumidora: se usan componentes originalmente desarrollados para otros proyectos.

Documentación en la codificación

Se considera como *documentación del programa* al conjunto de descripciones escritas que explican al lector qué hace el programa y cómo lo hace.

Se divide en

- Documentación interna: concisa, escrita en un nivel apropiado para el programador. Contiene información dirigida a quienes leerán el código fuente. Incluye información de algoritmos, estructuras de control y flujos.
- Documentación externa: se prepara para ser leída por quienes, tal vez, nunca verán código real como diseñadores o clientes.

Clase 9

Prueba de software

La etapa de prueba no es la primera instancia en que se localizan defectos. Se ha visto que la revisión de requerimientos y el diseño contribuyen a descubrir los problemas en las etapas tempranas.

Se entiende por software que ha fallado a aquel que no hace lo que especifican los requerimientos.

A veces sucede por

- Especificación errónea.
- Requerimientos imposibles con las estructuras previstas.
- Defectos en diseño del sistema.
- Defectos en código.

Tipos de defecto

- Algorítmicos: como no inicializar variables.
- De sintaxis: confundir un 0 por una O.
- De precisión: formulas implementadas incorrectamente.
- De documentación: no es acorde a lo que hace el software.
- De sobrecarga: el sistema funciona bien con 100 usuarios pero no con 110 por ejemplo.
- De capacidad: por ejemplo el sistema funciona bien con ventas < 1.000.000.
- De coordinación: por ejemplo la comunicación entre procesos tiene fallas.
- De rendimiento: tiempo de respuesta inadecuado.
- De recuperación: no volver a un estado normal luego de una falla.
- De relación hardware-software: incompatibilidad entre componentes.
- De estándares: no cumplir con la definición de estándares y procedimientos.

Clasificación ortogonal de defectos

Los defectos se organizan en categorías

En primer lugar se debe identificar si es un

1. Defecto por omisión: falta un aspecto clave en el código.
2. Defecto de cometido: un aspecto es incluido pero de forma incorrecta.

El objetivo de *diseñar pruebas* es que saquen a la luz diferentes clases de errores, con la menor cantidad de tiempo y esfuerzo.

Objetivos y beneficios de las pruebas del software

El objetivo es descubrir errores antes que el software salga del ambiente de desarrollo lo que permitirá bajar los costos de corrección en la etapa de mantenimiento.

Principios de la prueba

- Las pruebas deben planificarse mucho antes de que empiecen.
- Deben empezar por "lo pequeño" y seguir hacia "lo grande".
- Asegurarse que se aplican todas las condiciones a nivel componente.
- Las pruebas deben realizarse por un equipo independiente.

Se defiende la idea de un equipo independiente de pruebas para

1. Evitar el conflicto entre la responsabilidad por los defectos y la necesidad de descubrirlos.
2. Llevar a cabo las pruebas concurrentemente con la codificación.
3. Los desarrolladores deben colaborar y corregir los errores.

Tipos de prueba del software

Caja negra

Pruebas que se llevan a cabo sobre la interfaz del software y se centran en los requisitos funcionales del software.

Buscará errores de funciones incorrectas o ausentes, errores de interfaz, errores de estructuras de datos o bases de datos, rendimiento, inicialización y terminación.

Pruebas de partición equivalente

El diseño de los casos de prueba se basa en una evaluación de las clases de equivalencia para una condición de entrada.

Una clase de equivalencia representa un conjunto de estados válidos o no válidos para condiciones de entrada.

Una condición de entrada es un valor numérico específico, un rango de valores o un conjunto de valores relacionados.

- Si una condición de entrada especifica un rango, se define una clase de equivalencia válida y dos no válidas.
- Si una condición de entrada especifica un rango, se define una clase de equivalencia válida y dos no válidas.
- Si una condición de entrada requiere un valor específico, se define una clase de equivalencia válida y dos no válidas.
- Si una condición de entrada especifica un elemento de un conjunto, se define una clase de equivalencia válida y una no válida.
- Si una condición de entrada es lógica, se define una clase de equivalencia válida y una no válida

Análisis de valores límite (AVL)

Los errores tienden a darse más en los límites del campo de entrada que en el centro. Por eso, se ha desarrollado el análisis de valores límites como técnica de prueba.

Complementa a la partición equivalente. En lugar de seleccionar cualquier elemento de una clase de equivalencia, el AVL selecciona los casos de prueba en los extremos de la clase. En lugar de centrarse solamente en las condiciones de entrada, el AVL obtiene casos de prueba también para el campo de salida.

Análisis de valores límite

Para una condición de entrada de rango entre a y b probar: a, b, <a y >b.

Para una salida de rango entre a y b: usar casos de prueba que generen valor de salida a y b.

// queda pendiente ampliar

Caja blanca

Se basa en el minucioso examen de los detalles procedimentales. Se comprueban caminos lógicos poniendo casos de prueba que ejerciten conjuntos específicos de condiciones y/o bucles.

Deriva de casos de prueba de la estructura de control, para verificar detalles procedimentales. Mediante los métodos de prueba de caja blanca, el ingeniero de software puede obtener casos de prueba que garanticen que se ejercita por lo menos una vez todos los caminos independientes de cada módulo.

Ejercitará para asegurar su validez todas las decisiones lógicas, bucles en sus límites, estructuras internas de datos.

El flujo lógico de un programa a veces no es nada intuitivo, lo que significa que nuestras suposiciones intuitivas sobre el flujo de control y los datos nos pueden llevar a tener errores de diseño que solo se descubren cuando comienza la *prueba del camino*.

Es necesario realizarlas por los casos especiales que son más factibles de error, hay errores tipográficos aleatorios y el flujo de control intuitivo es distinto al real.

Prueba del camino básico

Técnica propuesta por Tom McCabe que permite al diseñador de casos de prueba obtener una medida de la complejidad lógica y usarla como guía para la definición de caminos de ejecución.

Los casos de prueba obtenidos garantizan que se ejecuta al menos una vez cada sentencia del programa.

// agregar gráficos y explicación

Pasos para crear la prueba del camino básico

1. Dibujar el grafo del flujo correspondiente.
2. Determinar la complejidad ciclomática.
3. Determinar un conjunto básico de caminos independientes.
4. Preparar los casos de prueba que forzarán la ejecución de cada camino del conjunto.
5. Ejecutar cada caso de prueba y comparar los resultados obtenidos con los esperados.

Clase 10

Enfoque estratégico de pruebas

Una estrategia de pruebas del software proporciona una guía que describe los pasos a seguir, cuándo se planean y llevan a cabo, cuánto esfuerzo, tiempo y recursos se requerirán.

Proporciona *planificación de las pruebas, diseño de los casos de prueba, ejecución de las pruebas, recolección y evaluación de los datos resultantes*.

La prueba es un conjunto de actividades que se planean con anticipación y se realizan de manera sistemática.

Conjunto de pasos en el que se incluyen técnicas y métodos específicos del diseño de casos de prueba.

Una estrategia de pruebas debe incluir pruebas de bajo y de alto nivel.

Las actividades de las estrategias de pruebas son parte de la *verificación* y *validación* incluidas en el aseguramiento de la calidad del software.

Concepto de verificación y validación

La **verificación** es el conjunto de actividades que asegura que el software implemente correctamente una función específica y la **validación** es un conjunto diferente de actividades que aseguran que el software construido corresponde con los requisitos del cliente.

Estrategias de pruebas

- Pruebas del sistema en la que se prueba el software como un todo.
- Pruebas de integración donde el foco de atención es el diseño y la arquitectura del software.
- Prueba de validación donde se validan los requisitos establecidos.
- Prueba de unidad donde comienza el espiral.
- Etapas al proceso de desarrollo.

// agregar espiral

Tipos de pruebas convencionales

- Pruebas de unidad: verifican que el componente funciona correctamente a partir del ingreso de distintos casos de prueba.

Se prueba la interfaz del módulo para asegurar que la información fluye de forma adecuada, examina estructuras de datos locales y pruebas las condiciones límite para asegurar que el módulo funciona correctamente además de ejercitar los caminos independientes.

En las pruebas de unidad los errores más comunes detectados son

1. Cálculos incorrectos como inicializaciones incorrectas, falta de precisión, representación simbólica incorrecta u operaciones mezcladas.
2. Comparaciones erróneas.
3. Flujos de control inapropiados.

Este tipo de casos de pruebas deben descubrir errores como

- Comparaciones entre diferentes tipos de datos.
- Operadores lógicos aplicados incorrectamente.
- Expectativas de igualdad como grado de precisión.
- Comparación incorrecta de variables.
- Terminación inapropiada o inexistente de bucles.
- Falla en la salida cuando se encuentre una iteración divergente.
- Variables de bucle modificadas inapropiadamente.

Para llevar a cabo estas pruebas, como un componente no es un programa independiente, se debe desarrollar para cada prueba de unidad un software que controle y/o resguarde.

Un controlador es un programa principal que acepta los datos del caso de prueba, pasa estos datos al módulo y muestra los resultados.

Un resguardo sirve para reemplazar a módulos subordinados al componente que hay que probar.

Los controladores y resguardos son una sobrecarga de trabajo. Si los controladores y resguardos son sencillos, el trabajo adicional es relativamente pequeño.

La prueba de unidad se simplifica cuando se diseña un módulo con un alto grado de cohesión.

- Pruebas de integración: verifican que los componentes trabajan correctamente en forma conjunta.

Se toman los componentes que han pasado las pruebas de unidad y se los combina según el diseño establecido.

En esta combinación es posible que

- Los datos se pueden perder en una interfaz.
- Un módulo puede tener un efecto adverso e inadvertido sobre otro.
- La combinación de subfunciones no produzca el resultado esperado.

El programa se construye y se prueba en pequeños segmentos en los que los errores son más fáciles de aislar y de corregir. La integración puede ser descendente o ascendente.

En la integración *descendente* los módulos se integran al descender por la jerarquía de control, iniciado por el programa principal. Podrá realizarse

- En profundidad: se integra todos los módulos de un camino de control principal de la estructura.
- En anchura: incorpora todos los módulos directamente subordinados a cada nivel.

// explicar mejor

En la integración *ascendente* se empieza la prueba con los módulos atómicos (es decir, módulos de los niveles más bajos de la estructura del programa). Dado que los módulos se integran de abajo hacia arriba, el proceso requerido de los módulos subordinados siempre está disponible y se elimina la necesidad de resguardos pero no así, los conductores

// explicar mejor

Los pasos son

1. Combinar módulos de bajo nivel.
2. Hacer conductor para coordinar entrada y salida.
3. Probar el grupo.
4. Eliminar conductores.

Hay discusión respecto a las ventajas y desventajas de los enfoques ascendente con descendente.

La principal desventaja del enfoque descendente es la necesidad de los resguardos.

La principal desventaja de la opción ascendente es que el programa como entidad no existe hasta que se agrega el último módulo.

La elección depende de las características del software, un emboque combinado puede ser el mejor.

Cada vez que se añade un nuevo módulo como parte de una prueba de integración, el software cambia.

Se establecen nuevos caminos, pueden ocurrir nuevas E/S y se invoca una nueva lógica de control. Estos cambios pueden causar problemas con funciones que antes trabajaban perfectamente.

En el contexto de una estrategia de prueba de integración, la *prueba de regresión* es volver a ejecutar un subconjunto de pruebas que se han llevado a cabo anteriormente para asegurarse de que los cambios no han propagado efectos colaterales no deseados.

Estas pruebas se pueden hacer manualmente, volviendo a realizar un subconjunto de todos los casos de prueba o usando herramientas automáticas.

El conjunto de pruebas de regresión contiene tres clases diferentes:

- Muestra representativa de pruebas que ejerciten todas las funciones del software.
- Pruebas adicionales que se centren en las funciones del software que son probablemente afectadas por el cambio.
- Pruebas que se centren en los componentes del software que han cambiado.

Se deben identificar los módulos críticos, que pueden ser los que

1. Abordan muchos requerimientos de software.
 2. Tienen alto nivel de control.
 3. Es complejo o proclive a error.
 4. Tiene requerimiento de rendimientos definidos.
- Pruebas de validación: proporcionan una seguridad final de que el software satisface todos los requisitos funcionales y no funcionales.

La validación del software se consigue mediante una serie de pruebas que demuestren la conformidad con los requisitos.

Una vez que se procede con cada caso de prueba de validación, puede darse una de las dos condiciones,

1. Las características de funcionamiento o de rendimiento están de acuerdo con las especificaciones y son aceptables.
2. Se descubre una desviación de las especificaciones y se crea una lista de deficiencias.

Este tipo de prueba inicia cuando finalizan las de integración. Habrá una revisión de la configuración donde hay que asegurar que todos los elementos de la configuración del software se hayan desarrollado apropiadamente, estén catalogadas y contengan detalle suficiente para reforzar la fase de soporte.

Habrá pruebas de aceptación (ALFA y BETA) que las realiza el usuario final que puede ser algo informal hasta la ejecución sistemática de una serie de pruebas bien planificadas.

Dentro de las pruebas de aceptación se pueden encontrar

- Pruebas ALFA: desarrolladores con clientes antes de liberar el producto. Se usa el software de forma natural con el desarrollador como observador y que irá registrando errores y problemas de uso.

Estas pruebas se hacen en un entorno controlado y se hacen luego de todos los procesos de pruebas básicas como unitarias o de integración.

No prueba la versión final de software y cierta funcionalidad puede ser añadida luego de las pruebas ALFA.

- Pruebas BETA: seleccionando los clientes que efectuarán la prueba. El desarrollador no está presente.

Se llevan a cabo por los usuarios finales del software en los lugares de trabajo de los clientes.

El desarrollador no está presente normalmente. Así, la prueba BETA es una aplicación en vivo del software en un entorno que no puede controlarse por el desarrollador.

El cliente registra los problemas que encuentra para luego informarlos.

BETA es la última fase de pruebas y se hace usando técnicas de caja negra.

A veces la versión beta también pueden ser liberados en el mercado y en base a las modificaciones existentes o no, se libera.

- Prueba del sistema: verifica que cada elemento encaja de forma adecuada y que se alcanza la funcionalidad y el rendimiento del sistema total.

La prueba está constituida por una serie de pruebas diferentes. Aunque cada prueba tiene un propósito diferente, todas trabajan para verificar que se han integrado adecuadamente todos los elementos del sistema y que realizan las funciones apropiadas.

Se clasifican en

- Pruebas de recuperación: controla la recuperación de fallas y el modo de reanudación del procesamiento en un tiempo determinado. Generalmente se fuerza el fallo para comprobarlo.
- Pruebas de seguridad: se comprueban los mecanismos de protección integrados.
- Pruebas de resistencia: se diseñan para enfrentar a los programas a situaciones anormales.
- Prueba de rendimiento: se prueba el sistema en tiempo de ejecución. A veces va emparejada con la prueba de resistencia.

Depuración

La depuración es el proceso de identificar y corregir errores en programas informáticos.

La depuración no es una prueba, pero siempre ocurre como consecuencia de una.

El proceso de depuración siempre tiene uno de los dos resultados:

1. Se encuentra la causa, se corrige y se elimina.
2. No se encuentra la causa. La persona que realiza la depuración debe sospechar la causa, diseñar un caso de prueba que ayude a confirmar sus sospechas y el trabajo vuelve hacia atrás a la corrección del error iterativamente.

Hay distintas **características de los errores**

- Síntoma lejano de la causa.
- Síntoma desaparece temporalmente a corregir otro error.
- Síntoma producido por error.
- Síntoma causado por error humano.
- Síntoma causado por problemas de tiempo.
- Condiciones de entrada difíciles de reproducir.
- Síntoma intermitente (especialmente en desarrollos hardware-software).
- El síntoma se debe a causas distribuidas entre varias tareas que se ejecutan en diferentes procesadores.

Enfoques de la depuración

- Diseñar programas de pruebas adicionales que repitan la falla original y ayuden a descubrir la fuente de la falla en el programa.
- Rastrear el programa manualmente y simular la ejecución.
- Usar las herramientas interactivas.
- Una vez corregido el error debe reevaluarse el sistema: volver a hacer las inspecciones y repetir las pruebas.

Prueba de entornos especializados

A medida que el software se hace más complejo, crece también la necesidad de enfoques de pruebas especializados.

- Pruebas de interfaces gráficas.
- Pruebas de arquitecturas cliente-servidor

- Prueba de servidor: probar las funciones de coordinación y manejo de datos del servidor.
 - Prueba de base de datos: probar la exactitud e integridad de los datos, examinar transacciones, asegurar que se almacenan, actualizan y recuperan datos.
 - Pruebas de transacciones: se crea una serie de pruebas para asegurar que cada transacción se procese de acuerdo a los requisitos.
 - Pruebas de comunicación de red: verifica comunicación entre los nodos, que el paso de mensajes, transacciones y tráfico de la red se realice sin errores.
 - Pruebas de la documentación y ayuda: es importante para la aceptación del programa, se revisa la guía del usuario o funciones de ayuda en línea. Las pruebas de documentación tiene dos fases
 - Revisar e inspeccionar la claridad editorial del documento.
 - Prueba en vivo, usar la documentación junto con el programa real.
 - Pruebas de sistema en tiempo real: el diseño de los casos de prueba, además de los convencionales deben incluir manejo de eventos, temporización de los datos, el paralelismo entre las tareas, etc.
 - Pruebas de tareas: probar de forma independiente, en búsqueda de errores lógicos y funcionamiento.
 - Pruebas de comportamiento: simular el comportamiento del sistema de tiempo real y lo examina como consecuencia de eventos.
 - Pruebas inter-tareas: se prueban las tareas asincrónicas entre las cuales se sabe que hay comunicación.
 - Pruebas de sistemas: se prueba el software y hardware integrados.
-

Clase 11

Mantenimiento

Es la atención del sistema a lo largo de su evolución después que el sistema se ha entregado. A esta fase se la llama *evolución del sistema*.

En ocasiones debe realizarse mantenimiento a sistemas heredados. En general estos sistemas se caracterizan por ser viejos, sin metodología ni documentación y sin modularización.

Es necesario evaluar cuándo es conveniente cerrar el ciclo de vida de ese sistema y reemplazarlo por otro.

La decisión se toma en función del costo del ciclo de vida del proyecto viejo y la estimación del nuevo.

En ocasiones la complejidad crece por los cambios.

El mantenimiento

- Soluciona errores.
- Añade mejoras.
- Optimiza.

Esto provoca costos adicionales y surge el fenómeno *barrera de mantenimiento*.

Características

- Su consecuencia es la disminución de otros desarrollos.
- Pueden existir efectos secundarios sobre código, datos y documentación.
- Las modificaciones pueden provocar disminución de la calidad total del producto.
- Las tareas de mantenimiento generalmente provocan reiniciar las fases de análisis, diseño e implementación.

- Involucra entre un 40% a 70% del costo total de desarrollo.
- Los errores provocan insatisfacción del cliente.

El mantenimiento resulta problemático ya que no es un trabajo atractivo, no siempre en el diseño se prevén cambios y es difícil comprender código ajeno, más aún sin documentación o que esta sea inadecuada.

Debe usarse un mecanismo para realizar los cambios que permite: identificarlos, controlarlos, implementarlos e informarlos.

El proceso de cambio se facilita si en el desarrollo están presentes los atributos calidad como modularidad, documentación interna del código fuente y de apoyo.

Ciclo de mantenimiento

1. Análisis: comprende el alcance y el efecto de la modificación.
2. Diseño: rediseñar para incorporar los cambios.
3. Implementación: recodificar y actualizar la documentación interna del código.
4. Prueba: revalida el software.
5. Actualizar la documentación de apoyo.
6. Distribuir e instalar nuevas versiones.

Facilidades en el desarrollo para ayudar al mantenimiento

- Análisis: señalar principios generales, armar planes temporales, especificar controles de calidad, identificar posibles mejoras, estimar recursos para mantenimiento.
- Diseño arquitectónico: claro, modular, modificable, con notaciones estandarizadas.
- Diseño detallado: notaciones para algoritmos y estructuras de datos, especificaciones de interfaces, manejo de excepciones, efectos colaterales.
- Implementación: indentación, comentarios de prólogo e internos, codificación simple y clara.
- Verificación: lotes de prueba y resultados.

Tipos de mantenimiento

- Mantenimiento correctivo: diagnóstico y corrección de errores.
- Mantenimiento adaptativo: modificación del software para interaccionar correctamente con el entorno.
- Mantenimiento perfectivo: mejoras al sistema.
- Mantenimiento preventivo: se efectúa antes que haya una petición, para facilitar el futuro mantenimiento. Se aprovecha el conocimiento sobre el producto.

Rejuvenecimiento del software

Es un desafío del mantenimiento, intentando aumentar la calidad global de un sistema existente.

Contempla retrospectivamente los subproductos de un sistema para intentar derivar la información adicional o reformarlo de un modo comprensible.

Los rejuvenecimientos podrán ser

- Re-documentación: representa un análisis del código para producir la documentación del sistema.
- Re-estructuración: se reestructura el software para hacerlo más fácil de entender.
- Ingeniería inversa: ante la ausencia de documentación, se analiza el código fuente para recuperar el diseño y, en muchos casos, reconstruir la especificación original del sistema. Este proceso es vital para comprender y

mantener sistemas heredados

- Re-ingeniería: es una extensión de la ingeniería inversa que produce un nuevo código fuente correctamente estructurado, mejorando la calidad sin cambiar la funcionalidad del sistema.
-

Auditoría informática

La *auditoría* es un examen crítico que se realiza para evaluar la eficiencia y eficacia de una sección o de un organismo y determinar cursos alternativos de acción para mejorar la organización y lograr los objetivos propuestos.

Puede ser interna, externa o una combinación.

Entonces, será la revisión y evaluación de los controles, sistemas y procedimientos de la informática, los equipos de cómputo y la organización que participa en el procesamiento de la información.

Permite definir estrategias para prevenir delitos, problemas legales, etc.

Es una actividad preventiva que el auditor sugiere y los procedimientos varían según la filosofía, técnicas y departamento de auditoría.

Objetivos

- Salvaguardar los activos.
- Integridad de datos.
- Efectividad de sistemas.
- Eficiencia de los sistemas.
- Seguridad y confidencialidad.

Influencia de la auditoría en informática

Algunos factores en la organización pueden influir mediante el control y la auditoría

- Controlar el uso de la computadora.
- Pérdida de capacidades de procesamiento de datos.
- Necesidad de mantener la privacidad individual.
- Posibilidad de pérdida de información o mal uso de la misma.
- Toma de decisiones incorrectas.
- Necesidad de mantener la privacidad de la organización.

Campo de acción

1. Evaluación administrativa del área de informática.
2. Evaluación de los sistemas y procedimientos, y de la eficiencia que se tiene en el uso de la información.
3. Evaluación del proceso de datos, de los sistemas y de los equipos de cómputo.
4. Seguridad y confidencialidad.
5. Aspectos legales de los sistemas y de la información.

